

Safety Analysis in Broadcast Networks Defined by Graph Grammars

Alpine Verification Meeting 2026

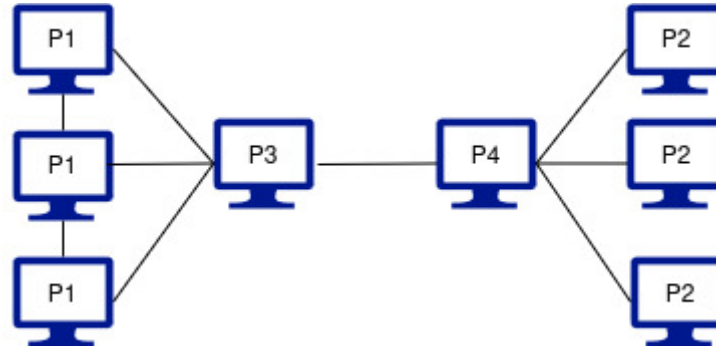
Christoffer Lind Andersen, Radu Iosif, Arnaud Sangnier

Verimag, CNRS

22/09/2026

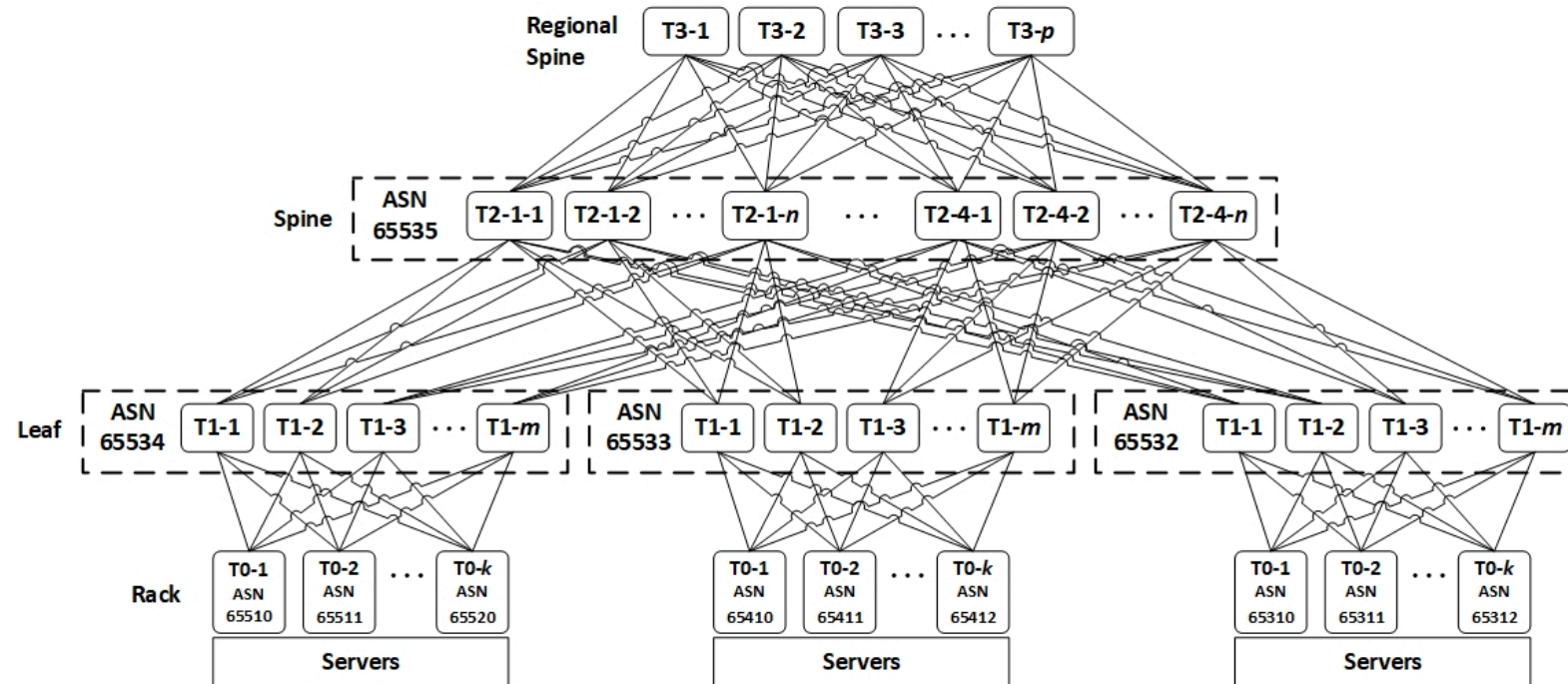
1. Introduction

1.1 Parameterized verification of distributed systems



- Underlying topology is unknown
- Size of the network is unknown
- Only communication by broadcast
- Communication schemes

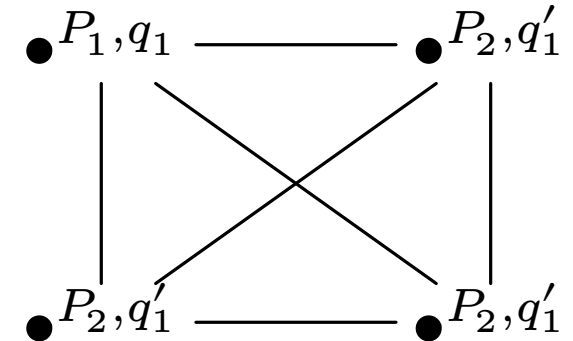
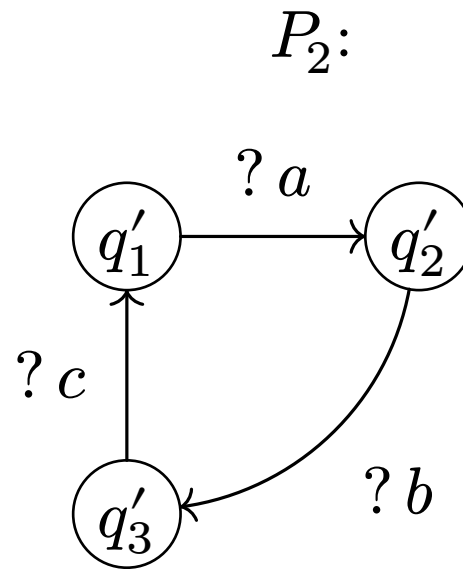
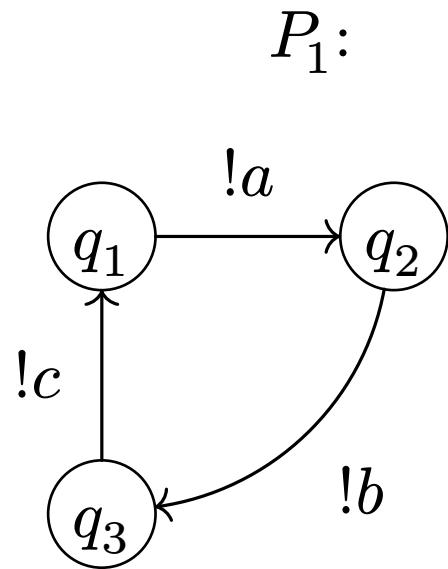
1.1 Parameterized verification of distributed systems



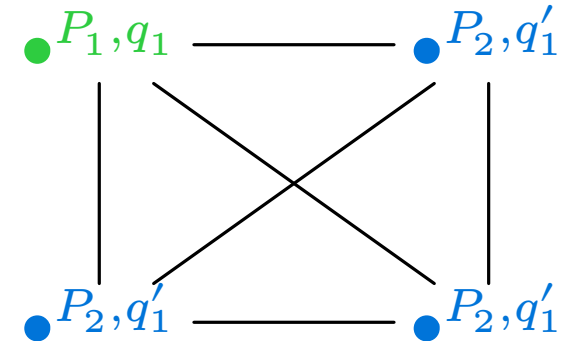
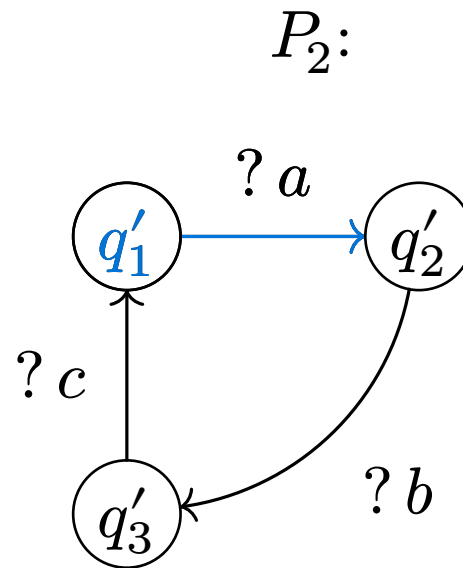
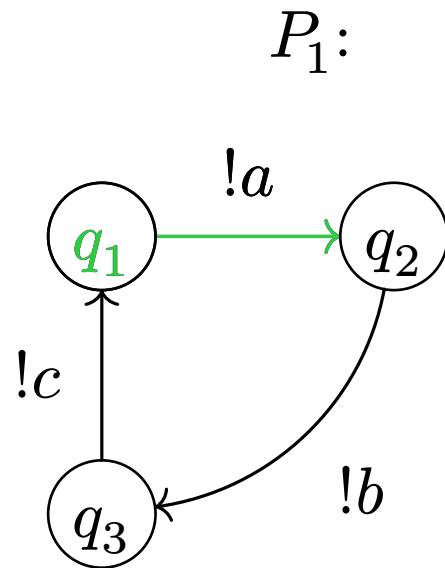
Jayaraman et al. 2019: Validating Datacenters At Scale

2. Model & Semantics

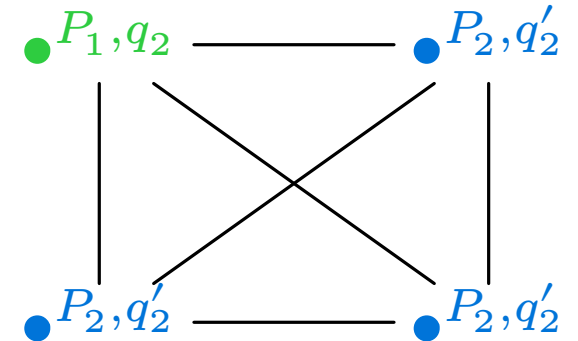
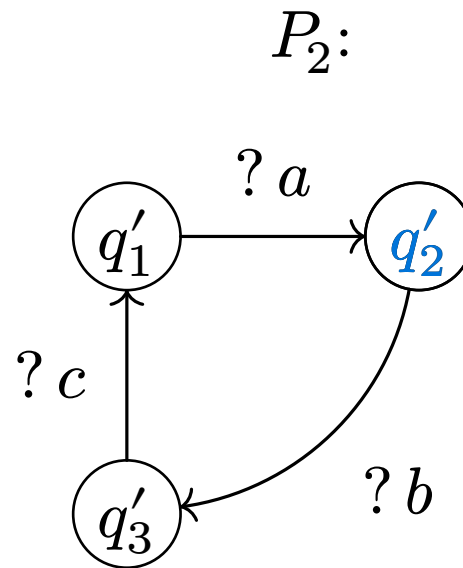
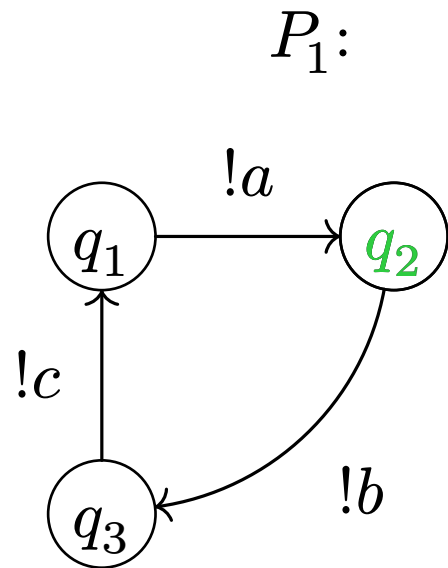
2.1 Process types



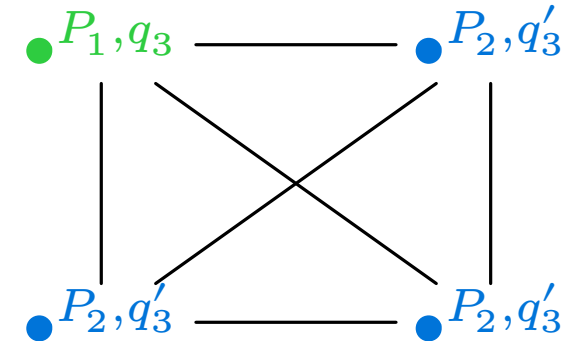
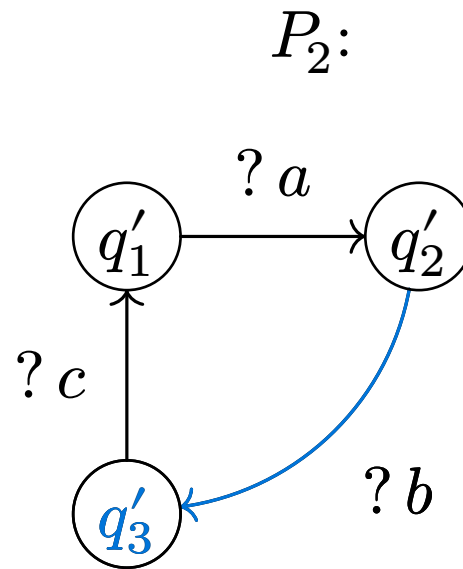
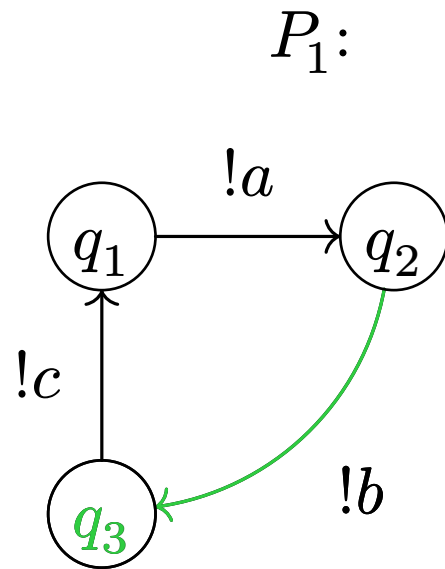
2.1 Process types

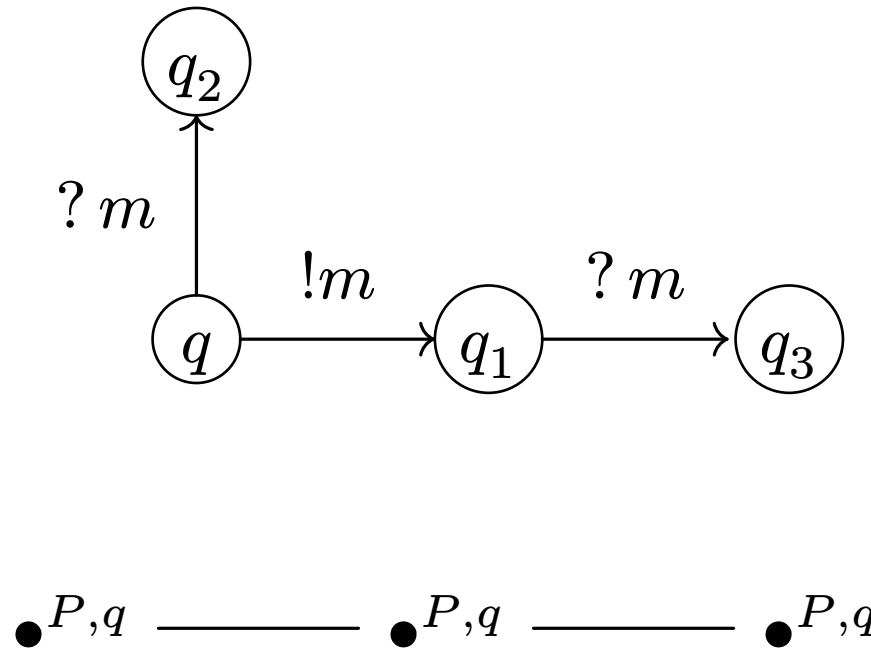


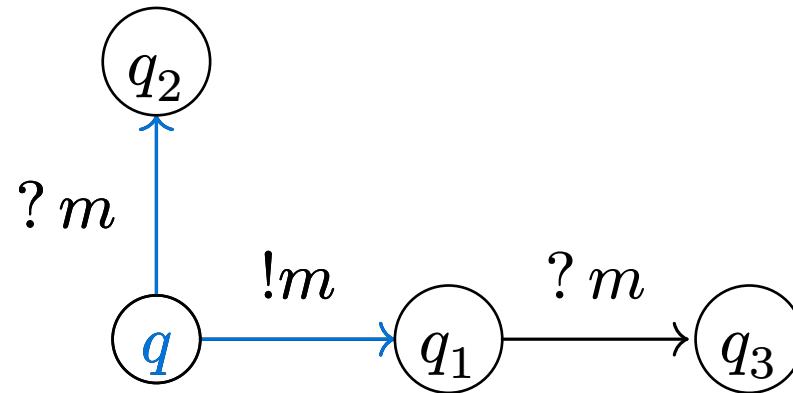
2.1 Process types



2.1 Process types

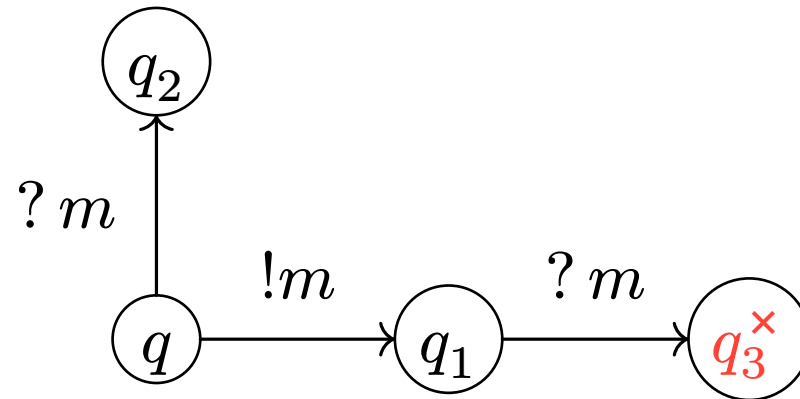




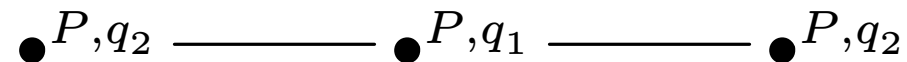


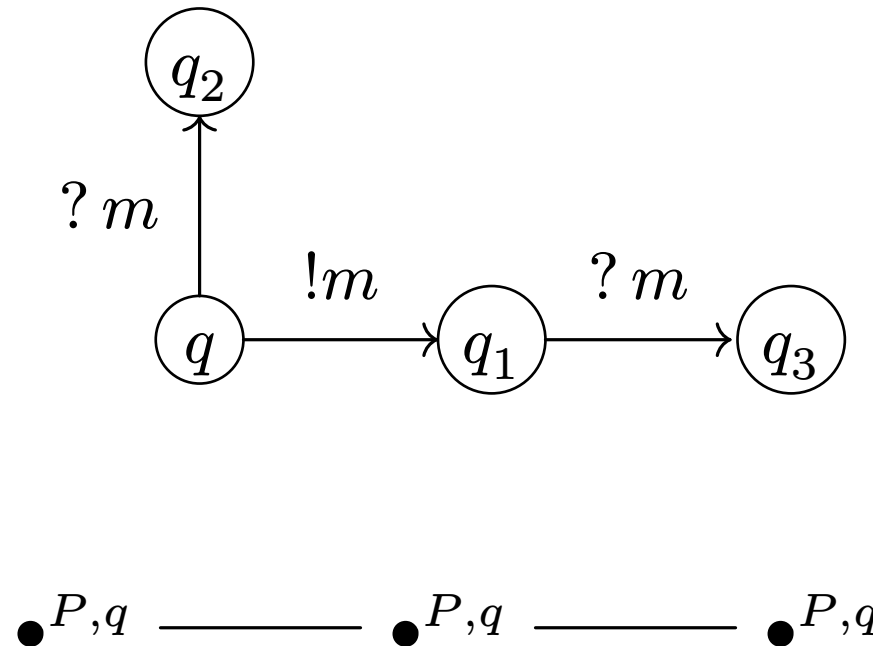
When we broadcast $!m$ we get:

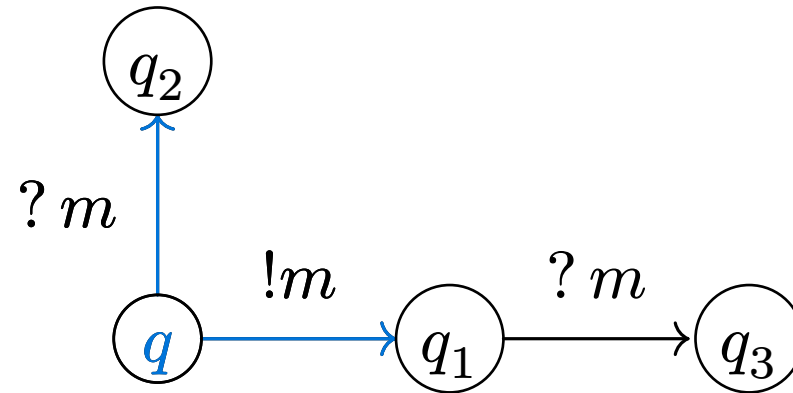




When we broadcast $!m$ we get:

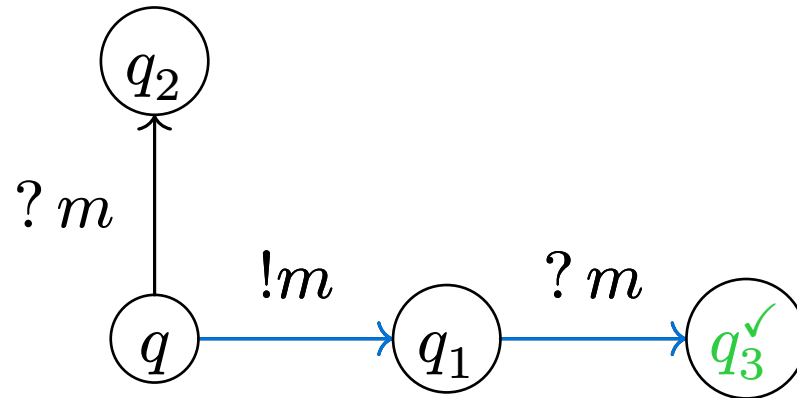






By having 1 process ignore $!m$ we can:



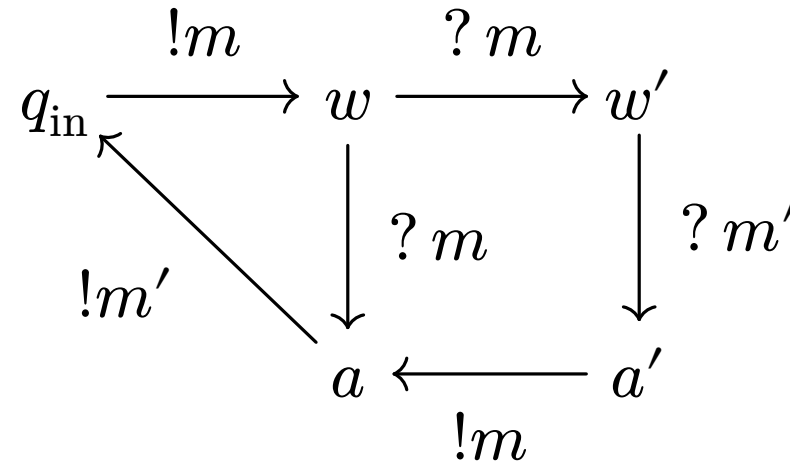
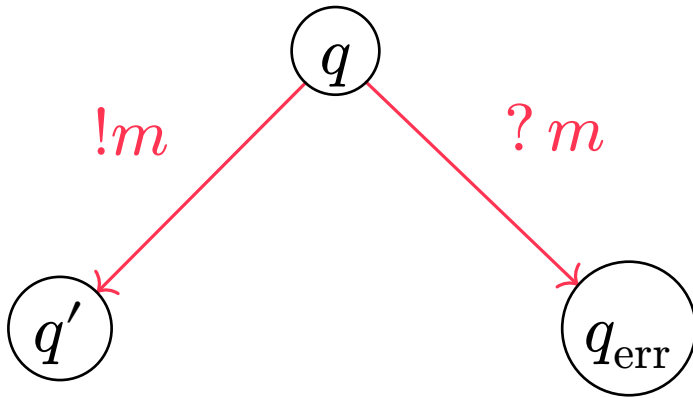


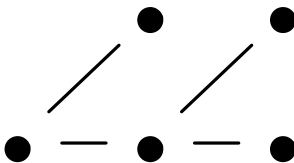
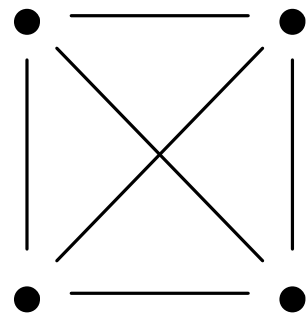
By having 1 process ignore $!m$ we can:



2.4 “Wait-only” restriction

1. a state is either an action state ($q \xrightarrow{!m} q'$) or a waiting state ($q \xrightarrow{?m} q'$)
2. q_{in} is always an action state





Network type	Communication	Coverability
Structured	Reliable	Undecidable ¹
Cliques	Reliable	Ackermann-hard ²
Cliques	Unreliable	PTime ³
Cliques	Wait-only Reliable	P-complete ⁴

¹Delzano et al. 2010

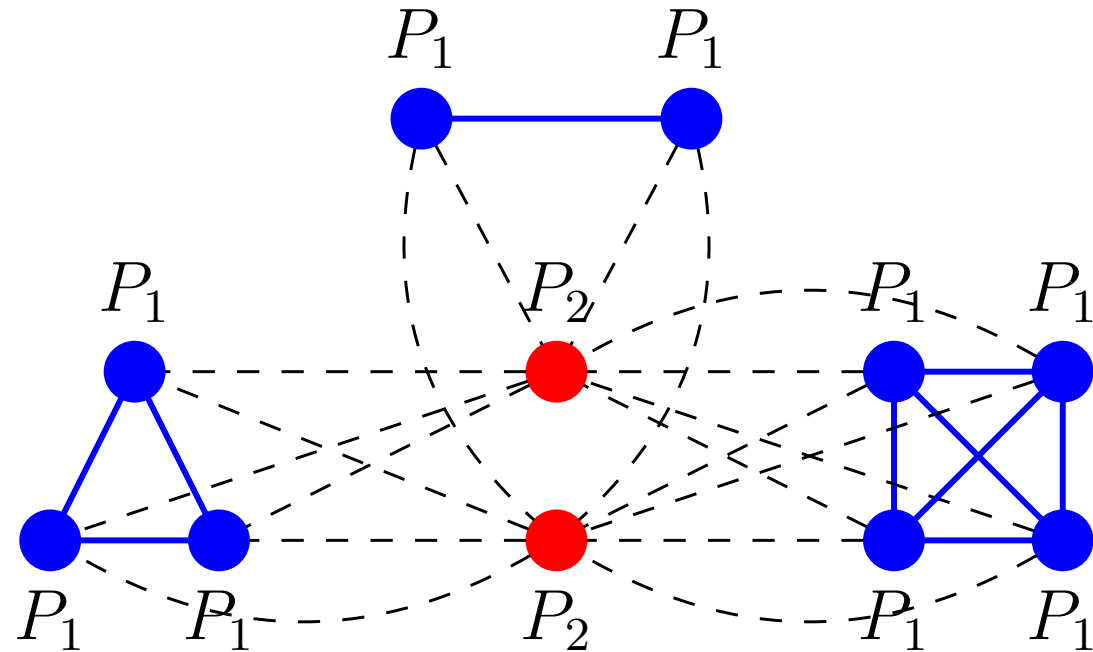
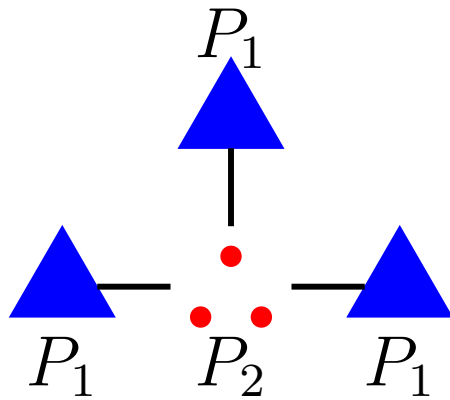
²Esparza et al. 1999

³Delzano et al. 2012

⁴Guillou et al. 2024

Network type	Communication	Coverability
Structured	Reliable	Undecidable
Cliques	Reliable	Ackermann-hard
Cliques	Unreliable	PTime
Cliques	Wait-only Reliable	P-complete
Clique-and-cloud	Reliable	Undecidable
Clique-and-cloud	Unreliable	Decidable
Clique-and-cloud	Wait-only Reliable	Decidable

3. Network topologies



We utilize graph grammars defined by using the vertex replacement operations recursively on rules to generate infinite families of graphs:

- Unit vertex with labels $G = \bullet_{\pi_1}^{P_1}$
- Disjoint union on graphs $G = G_1 \oplus G_2$
- Join 2 graphs with edges by label $G = \text{edg}_{\pi_1, \pi_2}(G')$
- Rename a label in the graph to something else $G = \text{relab}_{[\pi_1 \mapsto \pi_2]}(G')$

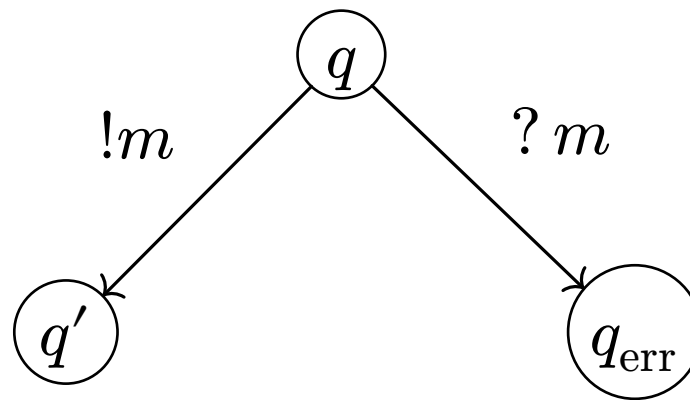
4. Verification

Input: A VR-graph grammar Γ and a control state q in the network

Question: Does there exist a graph $\mathcal{G}$ in the language of Γ where in one of its expansions we can reach q

- Easy for cliques - Finite graph i.e finite configuration
- Harder for clique-and-cloud graphs - Infinite graphs i.e infinite configurations

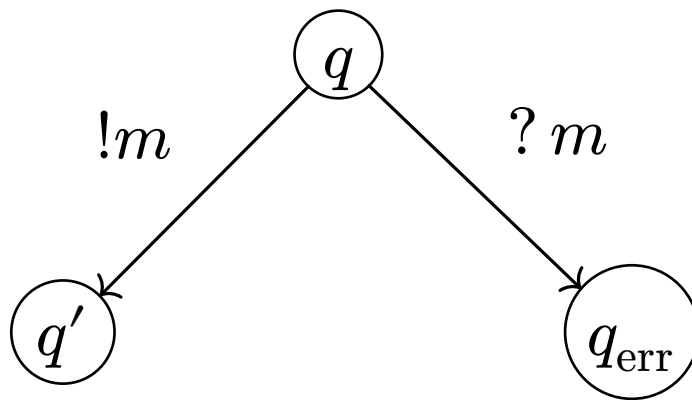
Lets consider putting a unique process type in each Δ :



In each clique we can now isolate a leader (1 in q' , rest in q_{err}):

$$\Delta_q^{C_2} \text{ --- } \Delta_q^{C_2} \text{ --- } \Delta_q^{C_2} \text{ --- } \Delta_q^{C_2} \text{ --- } \Delta_q^{\text{Ctl}} \text{ --- } \Delta_q^{C_1} \text{ --- } \Delta_q^{C_1} \text{ --- } \Delta_q^{C_1}$$

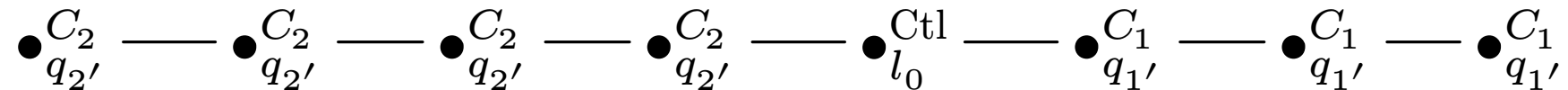
Lets consider putting a unique process type in each Δ :



In each clique we can now isolate a leader (1 in q' , rest in q_{err}):

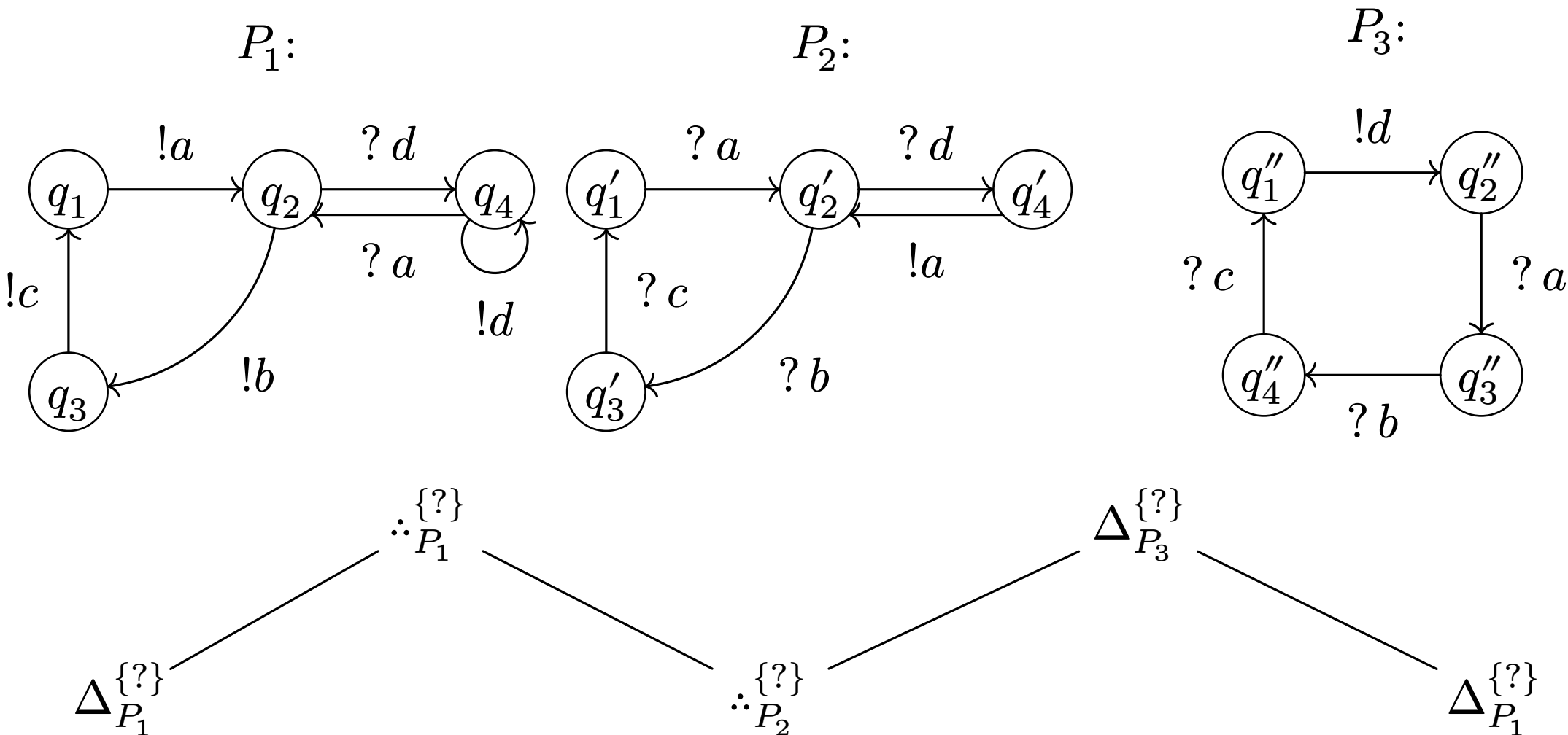
$$\Delta_{q'}^{C_2} \text{ --- } \Delta_{q'}^{C_2} \text{ --- } \Delta_{q'}^{C_2} \text{ --- } \Delta_{q'}^{C_2} \text{ --- } \Delta_{q'}^{\text{Ctl}} \text{ --- } \Delta_{q'}^{C_1} \text{ --- } \Delta_{q'}^{C_1} \text{ --- } \Delta_{q'}^{C_1}$$

We can now Simulate a 2-Counter machine with the line:

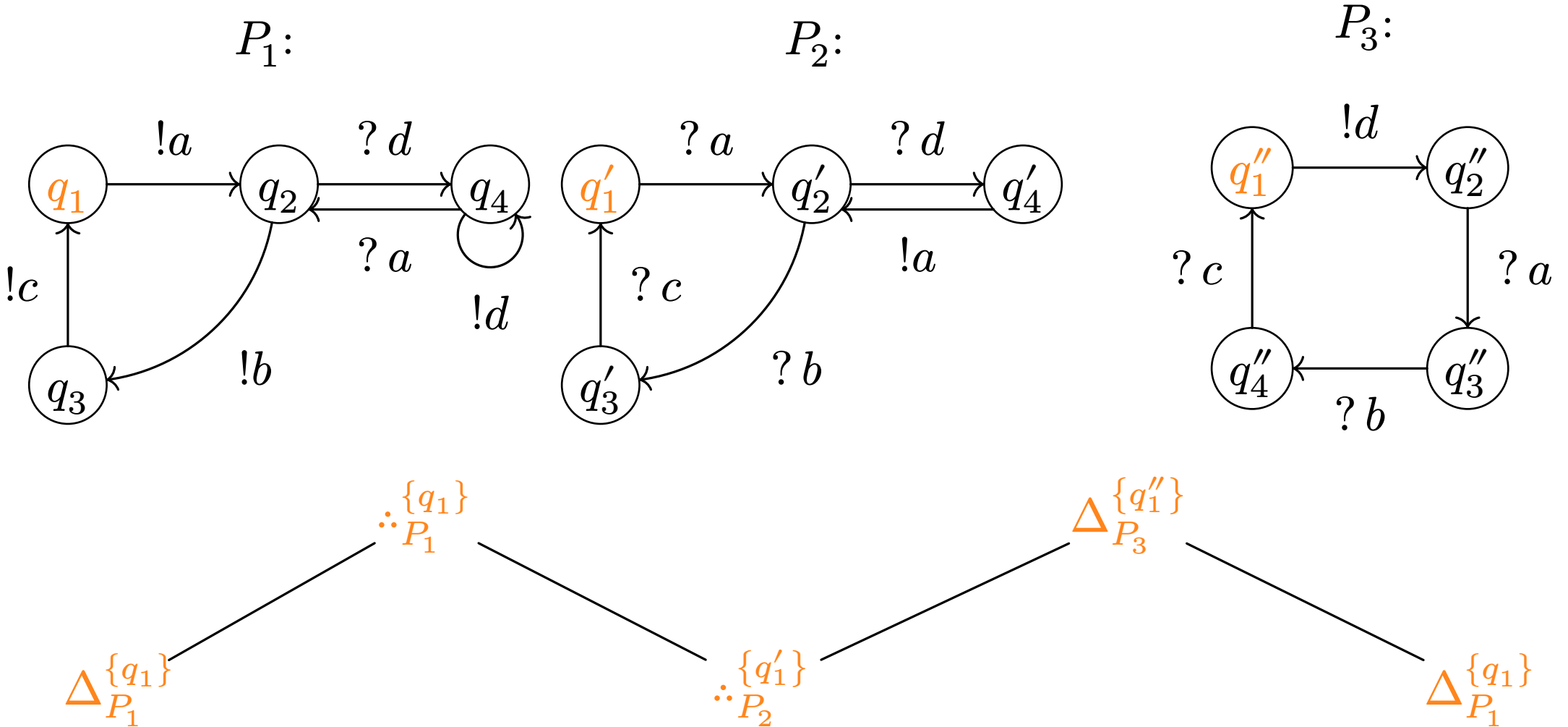


- We encode counting behaviour for C_1 and C_2
- We encode instruction behaviour in Ctl
- Result still holds when we consider clouds

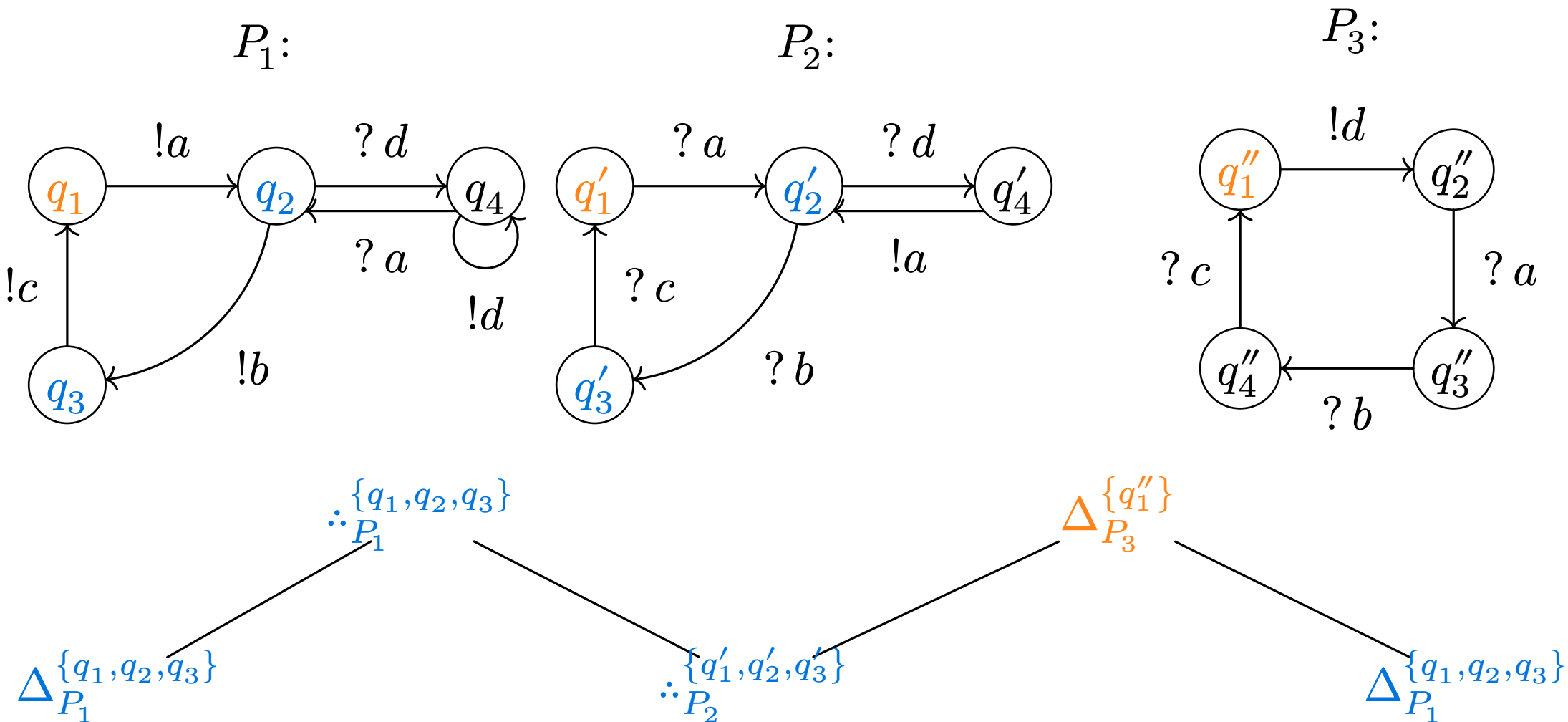
4.4 Reachable states labelling



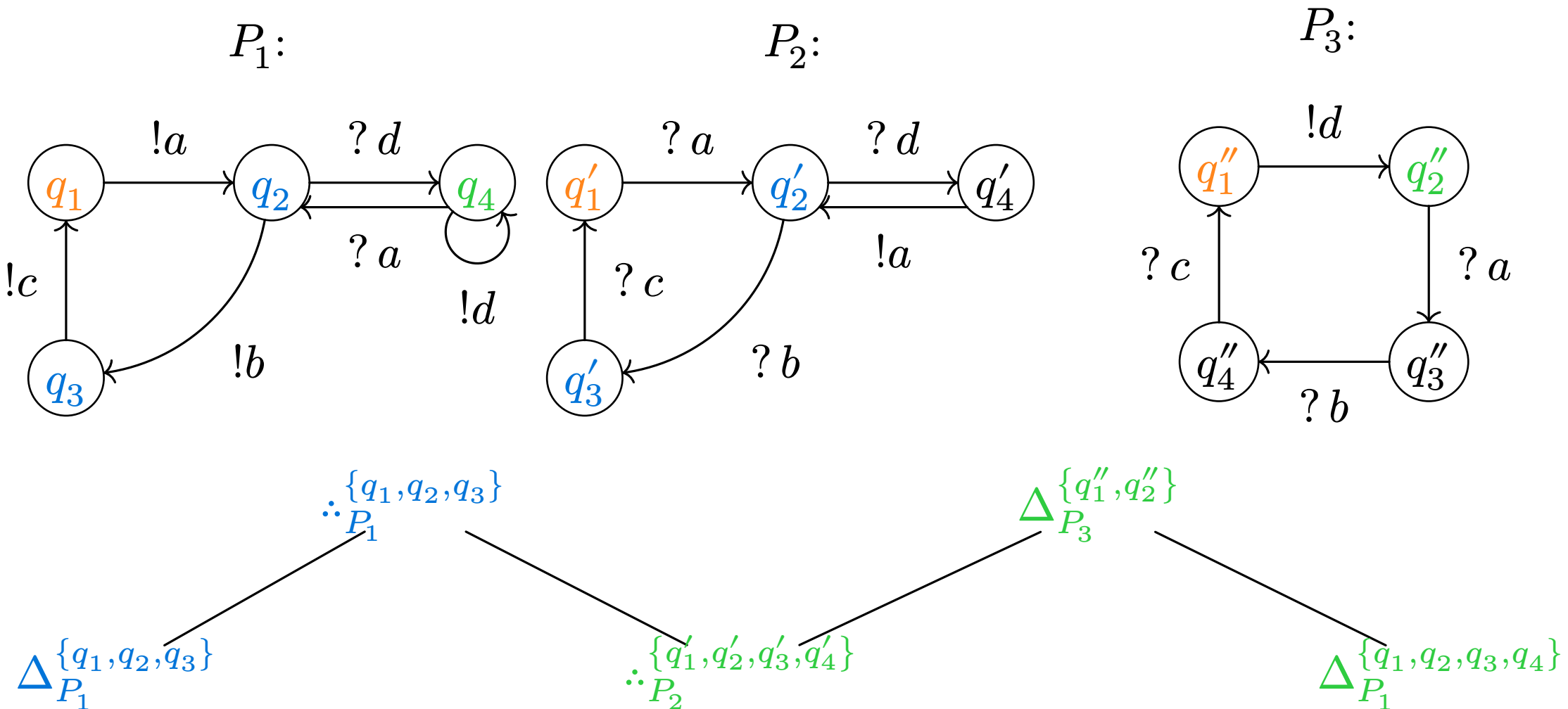
4.4 Reachable states labelling



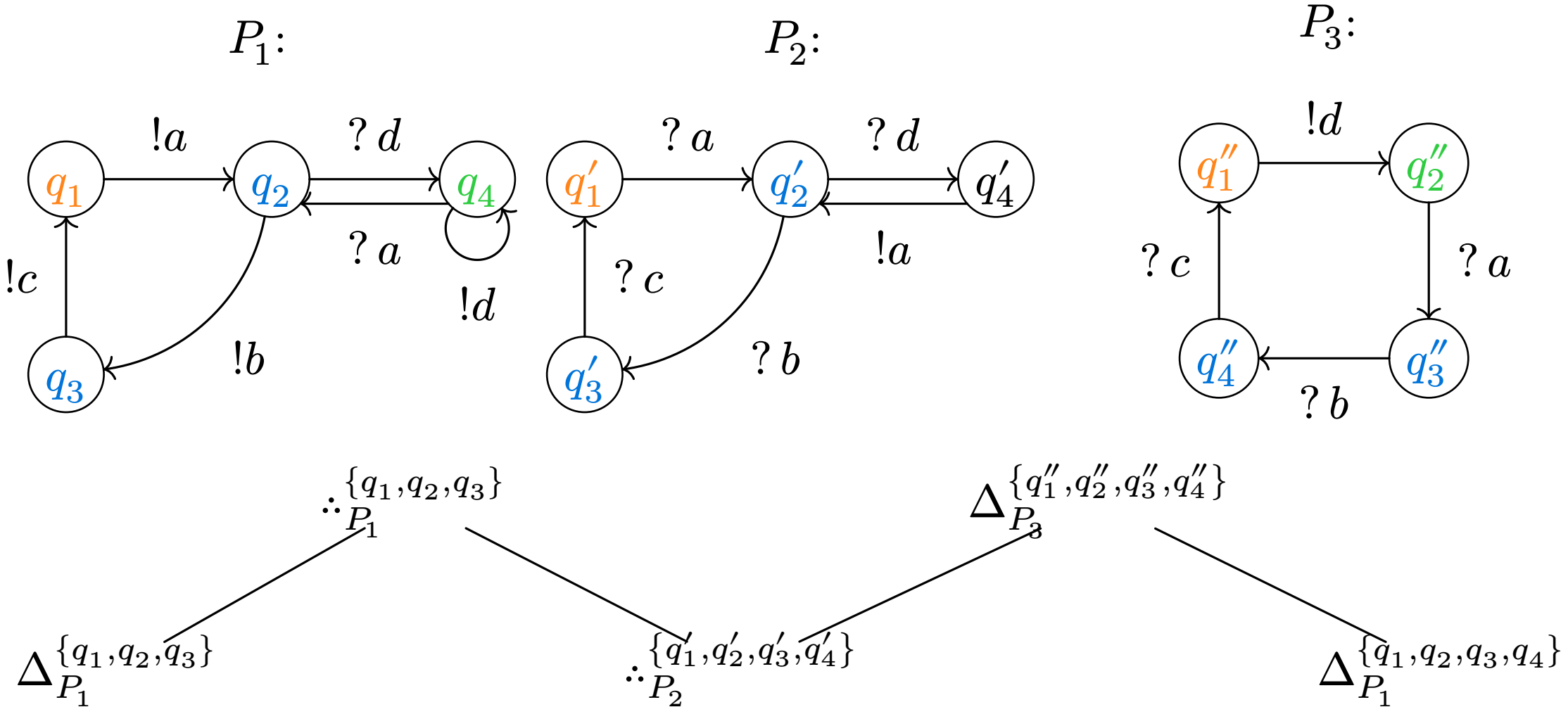
4.4 Reachable states labelling



4.4 Reachable states labelling



4.4 Reachable states labelling



Composition property for emitters:

If two configurations are reachable, then we can reach their joined configuration

Local successor property:

A configuration can only move if it broadcasts a message or receives from a neighbor.

To understand how processes move, consider the following execution:

$$q_1 \xrightarrow{!a} q_{1'} \xrightarrow{!b} q_{1''} \xrightarrow{!c} q_1$$

$$q_2 \xrightarrow{?a} q_{2'} \xrightarrow{?b} q_{2''} \xrightarrow{?c} q_2$$

Whatever the semantics we consider a process only moves when:

1. It makes a broadcast, or
2. It receives a message from its neighbors

Which we call the *local successor property*

The copy-cat property:

If a state is reachable, then we can have as many nodes in that state as we need

The copy-cat property:

If a state is reachable, then we can have as many nodes in that state as we need

If there is a vertex in G_1 in a state q then:

1. Unbounded expansion size $\Rightarrow$ Unbounded vertices in q
2. Joining with another $G_1 \uplus G_2$ remains in the same expansion

The copy-cat property:

If a state is reachable, then we can have as many nodes in that state as we need

If there is a vertex in G_1 in a state q then:

1. Unbounded expansion size $\Rightarrow$ Unbounded vertices in q
2. Joining with another $G_1 \uplus G_2$ remains in the same expansion

Note: this is possible due to the expansions and the fact that we can combine graphs $G_1 \uplus G_2$ and ignore messages

5. Decidability method

$$\varphi := \varphi \wedge \varphi \mid \neg \varphi \mid \exists x. \varphi \mid \exists X. \varphi \mid X(y) \mid E(x, y) \mid P_i(x) \mid \Delta(x) \mid \therefore (x)$$

$$\varphi := \varphi \wedge \varphi \mid \neg \varphi \mid \exists x. \varphi \mid \exists X. \varphi \mid X(y) \mid E(x, y) \mid P_i(x) \mid \Delta(x) \mid \therefore (x)$$

$$G \models_{\nu} \varphi$$

- Characterize verification problem in MSO
- Use the store ν as the states labelling
- Decidable for graph grammars:

The filtering theorem:¹

It is decidable whether there exists a graph G in $L(\Gamma)$ where $G \models \varphi$

¹Courcelle 1990

Consider the following MSO formula:

$$\exists x. \exists y. E(x, y) \wedge P_1(x) \wedge \Delta(x) \wedge P_2(y) \wedge \vdots(y)$$

$$\Delta^{P_1} \text{ ————— } \begin{matrix} P_2 \\ \vdots \end{matrix} \quad \Delta^{P_2} \text{ ————— } \begin{matrix} P_1 \\ \vdots \end{matrix} \quad \Delta^{P_1} \text{ ————— } \Delta^{P_3} \text{ ————— } \begin{matrix} P_2 \\ \vdots \end{matrix}$$

Consider the following MSO formula:

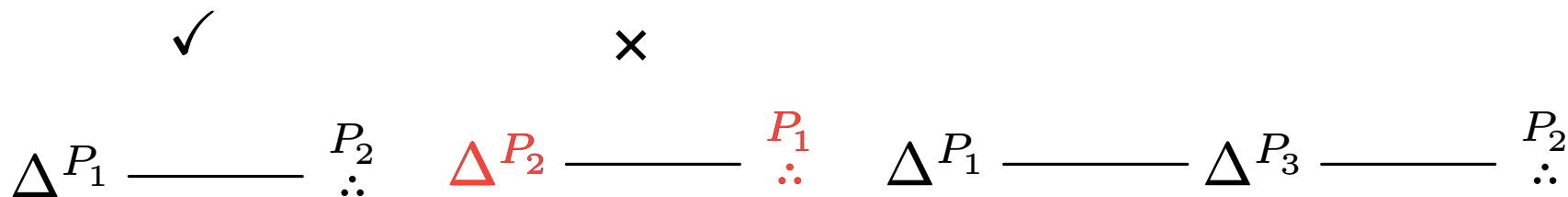
$$\exists x. \exists y. E(x, y) \wedge P_1(x) \wedge \Delta(x) \wedge P_2(y) \wedge \vdots(y)$$

✓

$$\Delta^{P_1} \text{ ————— } \begin{matrix} P_2 \\ \vdots \end{matrix} \quad \Delta^{P_2} \text{ ————— } \begin{matrix} P_1 \\ \vdots \end{matrix} \quad \Delta^{P_1} \text{ ————— } \Delta^{P_3} \text{ ————— } \begin{matrix} P_2 \\ \vdots \end{matrix}$$

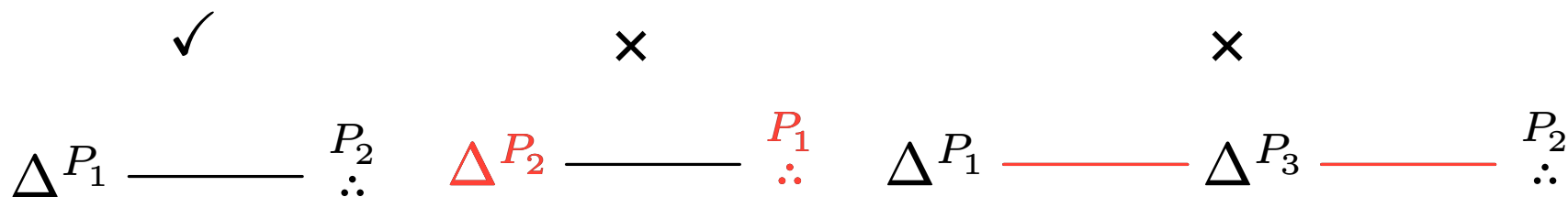
Consider the following MSO formula:

$$\exists x. \exists y. E(x, y) \wedge P_1(x) \wedge \Delta(x) \wedge P_2(y) \wedge \vdots(y)$$



Consider the following MSO formula:

$$\exists x. \exists y. E(x, y) \wedge P_1(x) \wedge \Delta(x) \wedge P_2(y) \wedge \vdots(y)$$



We build an MSO formula $\varphi_{\text{reach}}(X_1, \dots, X_n)$ such that:

- Each second order variable X_i corresponds to a control state q_i
- A vertex $v \in X_i$ iff it has processes that can reach q_i

Example of properties that this formula check:

- If $v \in X_i$ and there is $q_i \xrightarrow{!m} q_j$ then $v \in X_j$
- If $v \in X_i$ and $v \in X_k$ and there is $q_i \xrightarrow{?m} q_j$ and $q_k \xrightarrow{!m} q_l$ then $v \in X_j$
- If $v \in X_i$ and there is $q_i \xrightarrow{?m} q_j$ and v has a neighbor $v' \in X_k$ such $q_k \xrightarrow{!m} q'_l$ then $v \in X_j$
- The solution we find is the least solution

1. We model broadcast networks of different communication semantics
2. We consider infinite families of networks using VR-graph Grammars
3. Reachability analysis thanks to states labelling and MSO
4. Reachability in reliable broadcast is undecidable
5. We regain reachability for unreliable and wait-only networks

Next steps:

- Find complexity bounds of the problem
- Figure out new applications to the method (Target coverability)
- Liveness under fairness in cliques

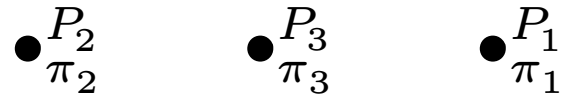
Questions?

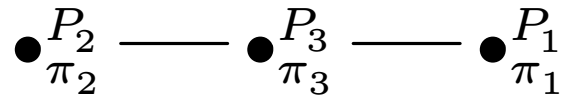
$$\begin{aligned} L &\hookrightarrow \text{edg}_{\pi_3, \pi_2}(\text{edg}_{\pi_3, \pi_1}(P_1 \oplus P_2 \oplus P_3)) \\ &\quad | \text{edg}_{\pi_1, \pi'_1}(\text{relab}_{[\pi_1 \mapsto \pi'_1, \pi'_1 \mapsto \emptyset]}(L) \oplus P_1) \\ &\quad | \text{edg}_{\pi_2, \pi'_2}(\text{relab}_{[\pi_2 \mapsto \pi'_2, \pi'_2 \mapsto \emptyset]}(L) \oplus P_2) \\ P_1 &\hookrightarrow \bullet \pi_1(K_1, \Delta) | \bullet \pi_1(K'_1, \Delta) | \bullet \pi_1(K''_1, \Delta) \\ &\quad | \bullet \pi_1(K_1, \cdot) | \bullet \pi_1(K'_1, \cdot) | \bullet \pi_1(K''_1, \cdot) \\ P_2 &\hookrightarrow \bullet \pi_2(K_2, \Delta) | \bullet \pi_2(K'_2, \Delta) | \bullet \pi_2(K''_2, \Delta) \\ &\quad | \bullet \pi_2(K_2, \cdot) | \bullet \pi_2(K'_2, \cdot) | \bullet \pi_2(K''_2, \cdot) \\ P_3 &\hookrightarrow \bullet \pi_3(Ctl, \Delta) | \bullet \pi_3(Ctl, \cdot) \end{aligned}$$

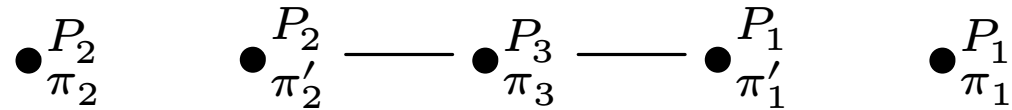
● P_3
 π_3

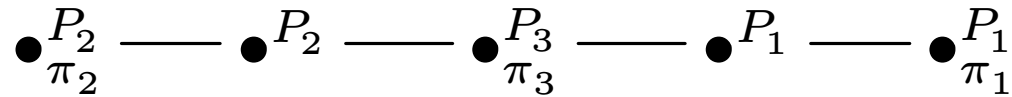
6.2 Vertex replacement grammars

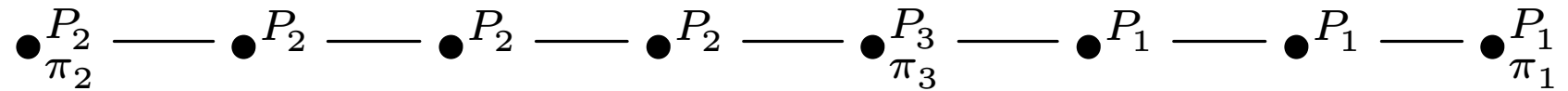
Questions?





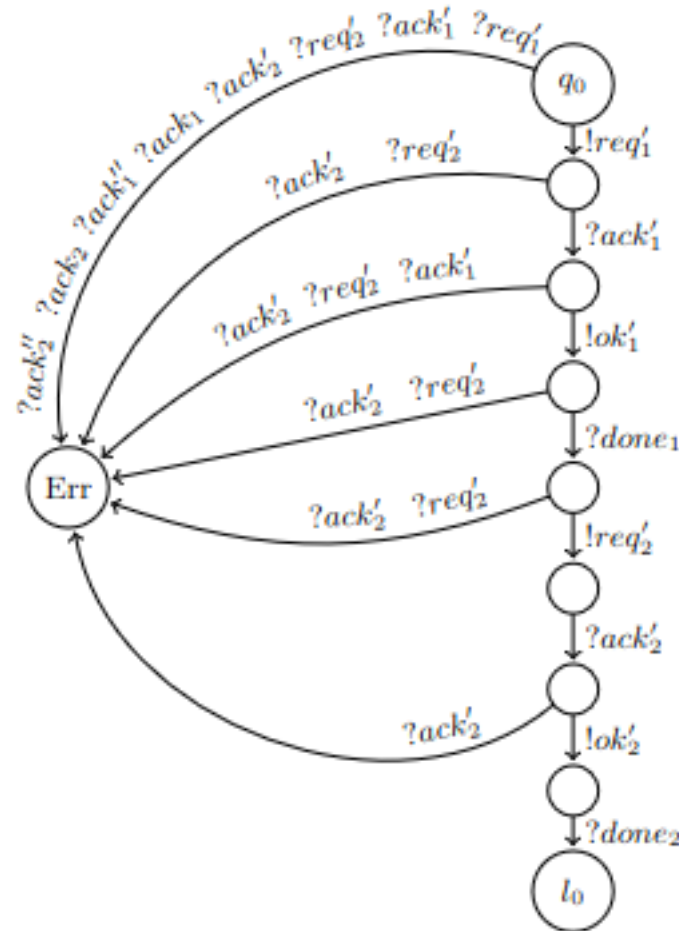






6.3 Leader selection in clouds

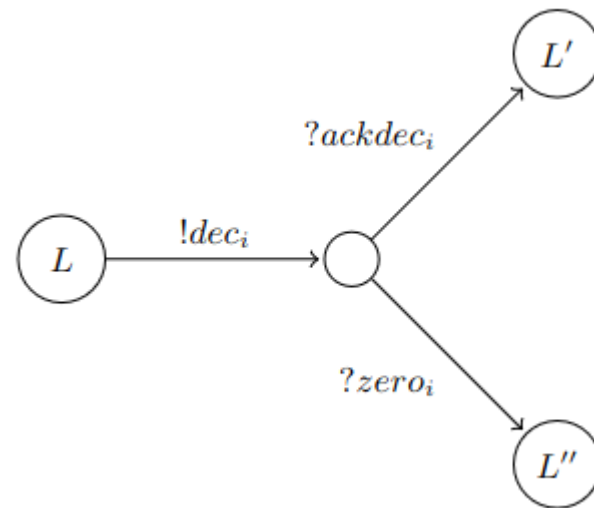
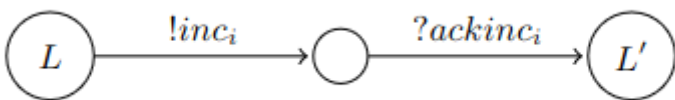
Questions?



6.4 Encoding a minsky machine

Questions?

Encoding increment and decrement instructions into process types:



6.4 Encoding a minsky machine

Questions?

